VOL. 50

LangChain

Building LLM-powered applications with the LangChain framework

Source	python.langchain.com/docs
Generated	2026-10-11
Level	Intermediate
Estimated study time	20 hours
Language	English
Volume	Vol. 50 - LangChain

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 4

Chapter 1 - What is LangChain? 4

1.1 Summary 4

1.2 Concepts 4

1.3 Examples 4

Example 1 - Basic chain 4

1.4 Mini test 5

Chapter 2 - Prompts and models 5

2.1 Summary 5

2.2 Concepts 5

2.3 Examples 6

Example 1 - ChatPromptTemplate with variables 6

2.4 Mini test 6

Chapter 3 - Memory 6

3.1 Summary 6

3.2 Examples 7

3.3 Mini test 7

Chapter 4 - Indexes and retrieval 7

4.1 Summary 7

4.2 Concepts 7

4.3 Examples 8

Example 1 - Retrieval chain 8

4.4 Mini test 8

Chapter 5 - Agents 8

5.1 Summary 8

5.2 Examples 9

5.3 Mini test 9

Chapter 6 - Where to go next 9

6.1 Summary 9

6.2 Cheat sheet 9

Final test 10

7.1 Final test answers 10

Sources **10**

How to use this volume

Prerequisites. Python programming, basic understanding of LLMs.

Estimated study time. About 20 hours.

Chapter 1 - What is LangChain?

1.1 Summary

- LangChain is a framework for building applications powered by language models.
- It provides a standard interface for models, prompts, and chains.
- Key concepts: models, prompts, indexes, memory, agents, chains.
- Install: `pip install langchain`.
- Most apps follow: load data → split → embed → retrieve → generate.

1.2 Concepts

Component	Purpose
Models	Wrapper around LLM APIs (OpenAI, Anthropic, etc.)
Prompts	Templates and prompt management
Indexes	Connect data sources and retrieve relevant context
Memory	Persist state across conversations
Chains	Sequences of calls to models and other components
Agents	Autonomous systems that use tools

1.3 Examples

Example 1 - Basic chain

Python

```python
# ch01_basic.py
# This code requires langchain installed
# It demonstrates the API structure

from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

# Define a prompt template
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    ("user", "{input}")
])

# Create an LLM
llm = ChatOpenAI(model="gpt-4o-mini", api_key="YOUR_KEY")

# Create a chain
chain = prompt | llm

# Run it (simulated output)
print("Chain structure:")
print("  prompt (ChatPromptTemplate)")
print("  ->")
print("  llm (ChatOpenAI gpt-4o-mini)")
print("  ->")
print("  result (model output)")

print("\nUsage:")
print("  result = chain.invoke({'input': 'Hello!'})")
```

1.4 Mini test

1. What is LangChain? 2. What does a prompt template do? 3. What is a chain?

Answers: 1) A framework for building LLM apps. 2) Manages dynamic prompts with placeholders. 3) A sequence of components working together.

Chapter 2 - Prompts and models

2.1 Summary

- Prompt templates allow parameterised, reusable prompts.
- System, human, and AI message types set different roles.
- Model wrappers standardise API calls across providers.
- Output parsers convert model output to structured formats.

2.2 Concepts

Type	Description
PromptTemplate	Basic template with variables

Type	Description
ChatPromptTemplate	Multi-message format (system + user + AI)
ChatOpenAI	Wrapper for OpenAI's chat API
StrOutputParser	Simple output parser that returns the string
PydanticOutputParser	Parses output into a Pydantic model

2.3 Examples

Example 1 - ChatPromptTemplate with variables

Python

```python
# ch02_prompt.py
from langchain_core.prompts import ChatPromptTemplate

# A template with placeholders
prompt_template = ChatPromptTemplate.from_messages([
    ("system", "You are a {role} assistant."),
    ("human", "Explain {concept} to {audience}."),
])

# Fill in the variables
messages = prompt_template.format_messages(
    role="patient",
    concept="recursion",
    audience="beginner programmers"
)

print("Generated messages:")
for msg in messages:
    print(f"  [{msg.type}]: {msg.content}")
```

output

```
Generated messages:
  [system]: You are a patient assistant.
  [human]: Explain recursion to beginner programmers.
```

2.4 Mini test

1. What does format_messages return? 2. What is StrOutputParser? 3. How to chain components?

Answers: 1) List of Message objects. 2) Returns the output as a raw string. 3) Using the | pipe operator.

Chapter 3 - Memory

3.1 Summary

- Memory allows the conversation to persist across interactions.
- Types: ConversationBufferMemory, ConversationSummaryMemory, EntityMemory.
- For long conversations, use summary-based memory.
- Memory can be stored externally (e.g., in a database).

3.2 Examples

Python

```python
# ch03_memory.py
from langchain.memory import ConversationBufferMemory, ConversationSummaryMemory

# Buffer memory stores all messages
buffer_memory = ConversationBufferMemory()

# Summary memory keeps a running summary (better for long conversations)
summary_memory = ConversationSummaryMemory(llm="...")

print("Memory types:")
print("  - ConversationBufferMemory: keeps entire history")
print("  - ConversationSummaryMemory: summarises conversation")
print("  - ConversationBufferWindowMemory: keeps last N messages")
print("  - EntityMemory: extracts and stores entities")
```

3.3 Mini test

1. What does ConversationBufferMemory do? 2. When to use summary memory? 3. How to add memory to a chain?

Answers: 1) Stores full conversation history. 2) For long conversations to save tokens. 3) Via the `memory` parameter of the chain.

Chapter 4 - Indexes and retrieval

4.1 Summary

- Indexes connect LLMs to external data sources.
- Key flow: load documents → split → embed → store in vectorstore → retrieve.
- Document loaders handle various file types.
- Text splitters chunk documents for embedding.
- Vector stores: FAISS, Chroma, Pinecone.

4.2 Concepts

Stage	Tool	Example
Load	UnstructuredFileLoader	Load PDFs, Word docs
Split	RecursiveCharacterTextSplitter	Split into chunks
Embed	OpenAIEmbeddings	Convert to vectors
Store	FAISS	Vector database
Retrieve	similarity_search	Find similar docs

4.3 Examples

Example 1 - Retrieval chain

Python

```python
# ch04_retrieval.py
# Requires: pip install faiss-cpu

from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

print("Retrieval chain structure:")
print("""
1. Load document
2. Split into chunks (e.g., 1000 chars, 200 overlap)
3. Embed chunks
4. Store in vector store
5. User asks question
6. Retrieve top-k similar chunks
7. Pass chunks + question to LLM
""")
```

4.4 Mini test

1. What is the purpose of a text splitter? 2. What does similarity search do? 3. What is a vector store?

Answers: 1) Break large documents into manageable chunks. 2) Find the most relevant chunks for a query. 3) A database that stores and searches vectors.

Chapter 5 - Agents

5.1 Summary

- Agents make decisions and use tools autonomously.
- The LLM decides what action to take (call a tool or respond).
- Tools: search, calculator, python, retriever, custom functions.

- The agent loops: decide → act → observe → decide.
- Tool definitions include: name, description, parameters.

5.2 Examples

Python

```python
# ch05_agents.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

print("Agent workflow:")
print("  1. LLM receives task and tools")
print("  2. LLM decides: respond or call a tool")
print("  3. If tool called: system executes tool, returns result")
print("  4. LLM receives result + context, decides again")
print("  5. Repeat until final response")

print("\nCustom tool example:")
print("""
@tool
def calculator(expression: str) -> float:
    \"\"\"Calculate a math expression.\"\"\"
    return eval(expression)

tools = [calculator]
llm_with_tools = ChatOpenAI().bind_tools(tools)
""")
```

5.3 Mini test

1. What does an agent decide at each step? 2. What is the agent loop? 3. How to define a custom tool?

Answers: 1) Whether to respond or call a tool. 2) Decide → Act → Observe → Repeat. 3) Use the @tool decorator with a description.

Chapter 6 - Where to go next

6.1 Summary

- Build a chatbot with memory.
- Build a research assistant with retrieval.
- Build an autonomous agent with tools.
- Explore: LangSmith for observability, LangServe for deployment.
- Read: LangChain documentation and cookbook examples.

6.2 Cheat sheet

Concept	Quick example
Prompt + LLM	`prompt | llm | parser`
RunnableParallel	`RunnableParallel(setup=..., query=...)`
Memory	`ConversationBufferMemory()`
Tool	`@tool` decorator
Agent	`create_react_agent(llm, tools)`

Final test

1. What is LangChain?
2. What does a prompt template do?
3. How do you chain components in LangChain?
4. What is ConversationBufferMemory?
5. What is the purpose of a text splitter?
6. What does bind_tools do?
7. What is the agent loop?
8. What does @tool do?
9. What is similarity_search used for?
10. What is the | operator used for?

7.1 Final test answers

1. A framework for building LLM-powered applications.
2. Creates customisable, parameterised prompts.
3. Using the | (pipe) operator.
4. Stores the full conversation history as context.
5. Break large documents into embeddable chunks.
6. Attaches tools to an LLM so it can decide to call them.
7. Decide → Act → Observe → Repeat until final response.
8. Decorator that converts a Python function into a LangChain tool.
9. Finding the most relevant chunks for a query.
10. Chaining components into a sequence (Runnable).

Sources

- https://python.langchain.com/docs
- https://python.langchain.com/docs/get_started/introduction
- https://python.langchain.com/docs/expression_language/
- https://python.langchain.com/docs/modules/agents